

Lecture 10

Unity로 체험하는 C#의 핵심 기능들

C# Feature Lab □ 작은 실험들의 모음

강영민

동명대학교 게임학부

목차

- 1 이 강의의 주인공은 C#
- 2 프로젝트 준비
- 3 Lab 1: enum + switch 식
- 4 Lab 2: 프로퍼티와 캡슐화
- 5 Lab 3: interface
- 6 Lab 4: event / Action
- 7 Lab 5: 제네릭 + LINQ
- 8 Lab 6: Coroutine + 람다
- 9 정리
- 10 도전 과제

Unity는 “무대” 다

이 과목의 주인공은 C#.

Unity는 `Console.WriteLine`보다 훨씬 친절한 무대일 뿐.

💡 콘솔 vs Unity

- 콘솔: `Console.WriteLine("HP: " + hp)` → 글자만
- Unity: `cube.color = Color.red` → 큐브가 빨개진다

같은 `int`, 같은 `enum`, 같은 `event`라도
눈으로 보면 머리에 더 잘 새겨진다.

이번 강의의 형식

큰 게임 한 편을 만들지 않는다.
대신 6개의 작은 실험(Lab)을 만든다.

각 Lab의 형식:

- ❶ 문제: “이런 상황이라면 어떻게?”
- ❷ 콘솔 비교: 일반 C# 콘솔이라면
- ❸ Unity 데모: 짧은 스크립트 + 키 입력
- ❹ 실험해 보기: 한 줄 바꾸고 결과 관찰

게임을 “완성”하는 것이 목표가 아니라
C# 기능 하나하나를 손에 익히기가 목표.

6개의 Lab

Lab	C# 기능
1	enum + switch 식
2	프로퍼티, 캡슐화
3	interface
4	event / Action<T>
5	제네릭 List<T> + LINQ
6	Coroutine + 람다

Unity Hub → New Project → 3D (또는 2D) → CSharpFeatureLab

씬에 미리 만들어 둘 GameObject:

- Main Camera, Directional Light (기본값)
- LabCube □ 3D Cube, (0,0,0) (Lab 1, 4에서 사용)
- LabRoot □ 빈 GameObject, Lab 스크립트들을 붙임
- Canvas + LogText (선택) □ 화면 위에 결과 표시

콘솔 창(Window → General → Console)을 열어 두고
Debug.Log와 **화면 변화를 동시에** 본다.

세 가지 “기분(mood)” 중 하나를 표현하고 싶다.

int 0/1/2로 쓰면...

- “state == 10이 무슨 뜻이지?”
- 오타도 잡히지 않음

→ 이름 있는 상수 집합이 필요

→ enum

Lab1_Enum.cs

```
1 public enum Mood { Happy, Sad, Angry }
2
3 public class Lab1_Enum : MonoBehaviour
4 {
5     [SerializeField] private Renderer targetRenderer;
6
7     void Update()
8     {
9         if (Input.GetKeyDown(KeyCode.Alpha1)) Apply(Mood.Happy);
10        if (Input.GetKeyDown(KeyCode.Alpha2)) Apply(Mood.Sad);
11        if (Input.GetKeyDown(KeyCode.Alpha3)) Apply(Mood.Angry);
12    }
13
14    void Apply(Mood m)
15    {
16        Color color = m switch
17        {
18            Mood.Happy => Color.yellow,
19            Mood.Sad   => new Color(0.3f, 0.5f, 1f),
20            Mood.Angry => Color.red,
21            -           => Color.white
22        };
23        targetRenderer.material.color = color;
24        Debug.Log($"Mood = {m}");
25    }
26 }
```


switch 표현식 (C# 8+)

```
1 // Old style (statement)
2 Color color;
3 switch (m) {
4     case Mood.Happy: color = Color.yellow; break;
5     case Mood.Sad:   color = Color.blue;   break;
6     default:        color = Color.white;  break;
7 }
8
9 // New style (expression)
10 Color color = m switch {
11     Mood.Happy => Color.yellow,
12     Mood.Sad   => Color.blue,
13     -          => Color.white
14 };
```

값을 반환하는 switch.
“무엇을 얻는가”가 한눈에.

Lab 1 Unity 구현 가이드

- 1 Hierarchy → 3D Object → Cube → 이름 MoodCube, 위치 (0,0,0)
- 2 Hierarchy → Create Empty → 이름 Lab1
- 3 Assets/Scripts/Lab1_Enum.cs 작성
- 4 스크립트를 Lab1 객체에 **드래그**해 부착
- 5 Inspector의 Target Renderer 슬롯에 MoodCube를 **드래그**
- 6 Play → 키 1, 2, 3

? 왜 GameObject 드래그가 Renderer로?

Unity가 자동으로 **그 GameObject의 Renderer 컴포넌트**를 찾아 채움.

```
= moodCube.GetComponent<Renderer>()
```

기초 문법: [SerializeField] private

- private 필드 = 같은 클래스 안에서만 접근
- Inspector에서도 안 보임 → 디자이너 친화적이지 않음
- [SerializeField] = “Inspector에 보여 주세요” 어트리뷰트

SerializeField 효과

[SerializeField] private float speed = 8f;

- 다른 클래스에서 lab.speed 접근 X
- Inspector에서 값 수정 O

[Foo] 형태는 모두 어트리뷰트

□ 클래스/필드/메서드에 “꼬리표” 붙이기

기초 문법: 문자열 보간 "\$..."

```
1 int n = 42;
2 string s = $"n = {n}";           // "n = 42"
3 string t = $"sum = {a + b}";     // expressions also work
4 string u = $"avg = {x:F2}";      // F2 = two decimal places
5 string v = $"{{score:D6}}";      // D6 = at least 6 digits, pad 0
```

- \$ 접두어 + {} 안에 식
- 콜론 뒤는 형식 지정자 (F2, D6 등)
- 옛 string.Format, "x = " + n보다 가독성 우수

실험해 보기 (Lab 1)

- `Mood.Excited`를 추가하고 키 4에 연결.
switch의 `_` 가지가 어떤 역할인지 확인
- `(int)Mood.Happy`는 무엇? `Debug.Log`로 출력
- `enum Mood : byte`로 바꾸면 무엇이 달라질까?
(저장 크기)

문제: 누구나 HP를 바꾸면?

```
1 class Character
2 {
3     public int hp; // BAD: anyone can write
4 }
5
6 var c = new Character();
7 c.hp = 100;
8 c.hp = -50; // negative is OK?
9 c.hp = 999999; // god mode
```

외부 읽기 OK, 외부 쓰기 차단이 필요.

→ `public int Hp { get; private set; }`

Lab2_Property.cs

```
1 public class Lab2_Property : MonoBehaviour
2 {
3     [SerializeField] private int maxHp = 10;
4     public int Hp { get; private set; } // <<< the key
5
6     void Start() { Hp = maxHp; }
7
8     void Update()
9     {
10         if (Input.GetKeyDown(KeyCode.D)) { Damage(1); Report(); }
11         if (Input.GetKeyDown(KeyCode.H)) { Heal(1); Report(); }
12
13         // The line below would NOT compile, by design:
14         // if (Input.GetKeyDown(KeyCode.R)) Hp = 9999;
15     }
16
17     public void Damage(int amount) => Hp = Mathf.Max(0, Hp - amount);
18     public void Heal(int amount) => Hp = Mathf.Min(maxHp, Hp + amount);
19
20     void Report() => Debug.Log($"HP = {Hp}/{maxHp}");
21 }
```

프로퍼티 한 줄 옵션들

☰ 프로퍼티 접근 수준

- { get; set; } □ 읽기 OK, 쓰기 OK (필드와 비슷)
- { get; private set; } □ 외부 읽기 OK, **쓰기 차단**
- { get; } □ **읽기 전용** (생성자에서만 초기화)

=>로 한 줄 메서드:

```
1 public void Damage(int n) => Hp = Mathf.Max(0, Hp - n);  
2
```


Lab 2 Unity 구현 가이드

- 1 Hierarchy → Create Empty → 이름 Lab2
- 2 Assets/Scripts/Lab2_Property.cs 작성, Lab2에 부착
- 3 Inspector에 **Max Hp = 10**이 보임
([SerializeField] 덕분)
Hp는 **보이지 않음** ([SerializeField]가 없는 자동 프로퍼티)
- 4 Window → General → Console
- 5 Play → **D**로 데미지, **H**로 회복

Console에 HP = 9/10, HP = 8/10...

Hp가 **0 미만/maxHp 초과로 가지 않음** (Mathf.Max/Min)

기초 문법: { get; private set; }을 풀어 보면

```
1 // short form (auto-property)
2 public int Hp { get; private set; }
3
4 // expanded by the compiler
5 private int _hp;           // hidden backing field
6 public int Hp
7 {
8     get { return _hp; }     // public read
9     private set { _hp = value; } // private write
10 }
```

- value는 set이 받는 값을 가리키는 **약속된 키워드**
- Hp = 10; → set의 value에 10

기초 문법: =>의 세 얼굴 (1/2)

C#의 =>는 문맥에 따라 세 가지로 쓰인다.
모두 “왼쪽이 들어가서 오른쪽이 나온다”.

(1) 식 본문 멤버 (Lab 2, Lab 3에서)

“이 메서드/프로퍼티는 이 식 한 줄을 반환”

```
1 public void Damage(int n) => Hp -= n;           // method
2 public string Name => "Circle";                 // property
3 public float Area() => Mathf.PI * r * r;         // method
```

(2) 람다 식 (Lab 5, Lab 6에서)

“매개변수를 받아 식을 반환하는 함수”

```
1 x => x >= 80                                     // Func<int, bool>
2 n => Debug.Log($"Tick {n}")                     // Action<int>
3 () => 42                                         // Func<int>
```

기초 문법: =>의 세 얼굴 (2/2)

(3) switch 식의 가지 (Lab 1에서)

“이 패턴이면 이 값”

```
1 Color c = m switch {  
2     Mood.Happy => Color.yellow,  
3     Mood.Sad   => Color.blue,  
4     -         => Color.white  
5 };
```

 기억하는 법: 어디 있는지를 보라

- 메서드/프로퍼티 선언 자리 → (1) 식 본문 멤버
- 인자/변수 자리에서 함수 자체를 만든다 → (2) 람다
- switch 안 → (3) 가지(arm)

실험해 보기 (Lab 2)

- `public int Hp { get; private set; } → public int Hp;`로 바꾸면?
다른 스크립트에서 `lab.Hp = 9999` 가능해진다
- `Hp { get; }`로 바꾸면?
Damage 안의 `Hp = ...`도 컴파일 에러
→ “어디까지 쓰기 허용”을 한 줄로 조절하는 감각

문제: 도형마다 면적 공식이 다르다

$$\text{Circle.Area()} = \pi r^2$$

$$\text{Rectangle.Area()} = w \cdot h$$

$$\text{Triangle.Area()} = \frac{1}{2}bh$$

공식은 다르지만, “**면적을 구할 수 있다**”라는 **능력**은 **공통**.

→ “**공통 능력**”만 약속해 두면

도형이 늘어나도 호출 코드는 그대로.

IShape와 두 구현체

```
1 public interface IShape
2 {
3     string Name { get; }
4     float Area();
5 }
6
7 public class CircleShape : IShape
8 {
9     private float radius;
10    public CircleShape(float r) { radius = r; }
11    public string Name => "Circle";
12    public float Area() => Mathf.PI * radius * radius;
13 }
14
15 public class RectangleShape : IShape
16 {
17     private float w, h;
18    public RectangleShape(float w, float h) { this.w = w; this.h = h; }
19    public string Name => "Rectangle";
20    public float Area() => w * h;
21 }
```

```
1 public class Lab3_Interface : MonoBehaviour
2 {
3     private IShape[] shapes;
4
5     void Start()
6     {
7         shapes = new IShape[] {
8             new CircleShape(2f),
9             new RectangleShape(3f, 4f),
10            new CircleShape(1.5f),
11        };
12    }
13
14    void Update()
15    {
16        if (Input.GetKeyDown(KeyCode.A))
17        {
18            foreach (var s in shapes)
19                Debug.Log($"{s.Name}: area = {s.Area():F2}");
20        }
21    }
22 }
```


interface의 핵심

interface 핵심

“무엇을 할 수 있는가”만 약속.

어떻게는 구현 클래스 자유.

상속 vs 인터페이스

- 상속: “Cat **is** Animal” (is-a)
- 인터페이스: “Cat **can** 울다” (can-do)

“면적을 **구할 수 있다**” → can-do → 인터페이스

Lab 3 Unity 구현 가이드

- 1 Hierarchy → Create Empty → 이름 Lab3
- 2 Assets/Scripts/에 4개 파일 생성:
IShape.cs, CircleShape.cs,
RectangleShape.cs, Lab3_Interface.cs
- 3 Lab3_Interface.cs만 Lab3에 드래그
(다른 셋은 MonoBehaviour가 아니므로 부착 불필요)
- 4 Window → Console, Play → A

i 파일명 = 클래스명 규칙

MonoBehaviour만 적용.

인터페이스/일반 클래스는 한 파일에 모아도 OK.

하지만 “한 파일 한 클래스”가 읽기 좋다.

기초 문법: public string Name => "Circle";

식 본문 프로퍼티(expression-bodied property)

“이 프로퍼티는 항상 이 식의 값을 반환”.

```
1 // Short form
2 public string Name => "Circle";
3
4 // Same meaning, expanded
5 public string Name
6 {
7     get { return "Circle"; }
8 }
```

- set이 없으니 외부 쓰기 불가
- 메서드 아니라 프로퍼티 → 괄호 없이 circle.Name
- =>의 세 얼굴 중 (1) 식 본문 멤버

기초 문법: foreach와 var

```
1 foreach (var s in shapes)
2     Debug.Log($"{s.Name}: {s.Area():F2}");
```

- `foreach (T x in xs)`: `xs`의 원소를 하나씩 `x`로
- `var s = “타입을 컴파일러가 알아서”`
`shapes`가 `IShape[]`이니 `s`는 자동으로 `IShape`

⚠ 주의

`var`는 “아무 타입이나”가 아니다.
타입은 컴파일 시점에 결정. JS의 `var`와 다름.

실험해 보기 (Lab 3)

- TriangleShape를 추가, shapes에 넣는다.
→ Lab3_Interface는 **한 줄도 수정 X**
- IShape에 Perimeter()를 추가하면?
모든 구현체가 **컴파일 에러**.
약속 위반은 컴파일러가 막아 준다.

문제: 변수가 바뀌면 “여러 곳”에 알려야

숫자 n 이 바뀔 때:

- UI 텍스트 갱신
- 사운드 재생
- 색상 변경
- ...

변수 쪽에서 **직접 호출**하면
알림 받는 곳이 늘 때마다 코드가 **비대해진다**.

- “소문”만 내고 듣는 사람은 알아서
- event

Lab4_Counter.cs (발신자)

```
1 using System;
2 using UnityEngine;
3
4 public class Lab4_Counter : MonoBehaviour
5 {
6     public event Action<int> OnChanged;
7     private int value_ = 0;
8
9     void Update()
10    {
11        if (Input.GetKeyDown(KeyCode.Space))
12        {
13            value_++;
14            OnChanged?.Invoke(value_);
15        }
16    }
17 }
```

발신자는 누가 듣는지 모른다.

?.Invoke: 구독자 없을 때 안전 호출.

Lab4_Logger / ColorChanger (구독자)

```
1 public class Lab4_Logger : MonoBehaviour
2 {
3     [SerializeField] private Lab4_Counter source;
4
5     void OnEnable() { source.OnChanged += React; }
6     void OnDisable() { source.OnChanged -= React; }
7
8     void React(int v) => Debug.Log($"Logger heard: {v}");
9 }
10
11 public class Lab4_ColorChanger : MonoBehaviour
12 {
13     [SerializeField] private Lab4_Counter source;
14     [SerializeField] private Renderer target;
15
16     void OnEnable() { source.OnChanged += React; }
17     void OnDisable() { source.OnChanged -= React; }
18
19     void React(int v)
20     {
21         float t = (v % 10) / 10f;
22         target.material.color = Color.Lerp(Color.black, Color.cyan, t);
23     }
24 }
```


delegate, Action, event 한 묶음

- delegate: 메서드를 변수처럼 담는 그릇
- Action<T>: “T 받고 void” 미리 만들어 둔 delegate
- Func<T, R>: “T 받고 R 반환”
- event: 외부에서 +=/-=만 가능 (직접 호출 X)
- ?.Invoke(...): null 안전 호출

⚠ 필수 짝

OnEnable에서 +=, OnDisable에서 -=.
안 그러면 메모리 누수 + 파괴된 객체에 호출.

Lab 4 Unity 구현 가이드 (1/2)

이번 Lab은 여러 컴포넌트가 서로 참조 → 단계가 길다.

Hierarchy 구조:

```
Lab4_Source      <-- Lab4_Counter (발신자)
Lab4_Subscribers <-- Lab4_Logger
                  Lab4_ColorChanger
Lab4_Cube         <-- 3D Cube (색이 바뀜)
```

- 1 Lab4_Source (Empty) + Lab4_Counter.cs
- 2 Lab4_Cube (3D Cube), 위치 (2, 0, 0)
- 3 Lab4_Subscribers (Empty)
+ Lab4_Logger.cs, Lab4_ColorChanger.cs
(한 객체에 컴포넌트 두 개)

Lab 4 Unity 구현 가이드 (2/2)

Inspector 슬롯 연결 (가장 중요):

- `Lab4_Logger.Source` ← `Lab4_Source` 드래그
(Unity가 `Lab4_Counter` 컴포넌트를 자동 추출)
- `Lab4_ColorChanger.Source` ← `Lab4_Source`
- `Lab4_ColorChanger.Target` ← `Lab4_Cube`

Play → **SPACE**

콘솔에 **로그 + 큐브 색이 동시에 변함.**

두 구독자는 서로의 존재를 **모른다.**

기초 문법: Action<T>, Func<T, R>

```
1 // Pre-defined delegates in .NET
2 public delegate void Action<T>(T arg);
3 public delegate R Func<T, R>(T arg);
```

- Action □ 매개변수 없음, void 반환
- Action<int> □ int 받고 void
- Action<int, string> □ int+string 받고 void
- Func<int, bool> □ int 받고 bool 반환
- Func<int, int, int> □ 마지막으로 반환 타입

매번 `public delegate void MyHandler(int);`
을 새로 정의할 필요가 없게 해 주는 단축.

기초 문법: ?.와 ?.Invoke(...)

```
1 string s = null;
2 int len = s.Length;    // NullReferenceException!
3 int? len2 = s?.Length; // if s is null, len2 is null. Safe.
4
5 // Equivalent two lines
6 OnChanged?.Invoke(value_);
7 if (OnChanged != null) OnChanged(value_);
```

- ?. □ “왼쪽이 null이 아닐 때만 오른쪽”
- event는 외부에서 직접 호출 불가 → Invoke()
- ?.Invoke(...) = 사실상 표준 패턴

기초 문법: OnEnable/OnDisable 시점

Lecture 9의 MonoBehaviour 생명 주기 중 두 개:

- **OnEnable** □ 컴포넌트 **활성화될 때**
Play 시작, 체크박스 켜기, `SetActive(true)`
- **OnDisable** □ 컴포넌트 **비활성화될 때**
체크박스 끄기, `SetActive(false)`, 객체 파괴 직전

Start/OnDestroy vs OnEnable/OnDisable

Start/OnDestroy로 짤 맞춰도 동작은 함.
하지만 **컴포넌트가 켜졌다 꺼졌다** 할 때
OnEnable/OnDisable이 **자동으로 호출**되어 더 안전.

문제: 점수들의 통계를 빠르게

여러 점수를 모아 두고...

- Count, Sum, Average, Max, Min
- “80점 이상이 몇 명?”

배열은 크기 고정 → `List<T>`

for 루프 직접? → LINQ 한 줄로

같은 `List` 클래스가 `int`, `string`, `Mood`...

무엇이든 담을 수 있다 → 제네릭

Lab5_Generic.cs

```
1 using System.Collections.Generic;
2 using System.Linq;
3 using UnityEngine;
4
5 public class Lab5_Generic : MonoBehaviour
6 {
7     private List<int> scores = new List<int>();
8
9     void Update()
10    {
11        if (Input.GetKeyDown(KeyCode.N))
12        {
13            int s = Random.Range(0, 100);
14            scores.Add(s);
15            Debug.Log($"Added: {s}, Count={scores.Count}");
16        }
17        if (Input.GetKeyDown(KeyCode.L))
18        {
19            if (scores.Count == 0) { Debug.Log("No scores yet."); return; }
20            int sum = scores.Sum();
21            double avg = scores.Average();
22            int top = scores.Where(x => x >= 80).Count();
23            Debug.Log($"Sum={sum}, Avg={avg:F1}, >=80: {top}");
24        }
25    }
26 }
```


제네릭 <T>: 타입을 변수처럼

- `List<int>`, `List<string>`, `List<Mood>`...
- `List<int>.Add("hi")`는 **컴파일 에러**
(옛 `ArrayList`는 런타임에야 발견)

LINQ: `using System.Linq;`로 마법 메서드들

- `.Sum()` `.Average()` `.Max()` `.Count()`
- `.Where(predicate)` □ 거르기
- `.Select(transform)` □ 변환
- `.OrderBy(key)` □ 정렬

`x => x >= 80`: **람다** (`Func<int,bool>`)

Lab 5 Unity 구현 가이드

❶ Hierarchy → Create Empty → 이름 Lab5

❷ Lab5_Generic.cs 작성 후 Lab5에 부착

❸ 파일 맨 위 **using** 두 줄 필수:

```
using System.Collections.Generic; // List<T>  
using System.Linq;                // .Sum(), .Where()
```

❹ Console 열기, Play → **N/L/K**

- **N** □ 점수 추가 (0~99 랜덤)
- **L** □ 통계 (Sum/Avg/≥80)
- **K** □ 비우기

기초 문법: 람다 $x \Rightarrow x \geq 80$

Where는 `Func<int, bool>`을 인자로 요구.

```
1 IEnumerable<int> Where(Func<int, bool> predicate);
2
3 // Old way: define a method, pass as method group
4 bool IsHigh(int x) { return x >= 80; }
5 scores.Where(IsHigh);
6
7 // New way: lambda directly
8 scores.Where(x => x >= 80);
```

컴파일러가 x 는 `int`, 반환은 `bool`로 자동 추론.

람다의 모양들:

```
1 x => x + 1           // single param: parentheses optional
2 () => 42              // no params
3 (a, b) => a + b       // multiple params
4 x => { Debug.Log(x); } // block body
```

기초 문법: using System.Linq; = 확장 메서드

List<int>에는 Sum, Where가 **없다**.

그런데 using System.Linq;만 쓰면 사용 가능?

```
1 namespace System.Linq
2 {
3     public static class Enumerable
4     {
5         public static int Sum(this IEnumerable<int> source) { ... }
6     }
7 }
```

핵심은 첫 매개변수의 **this** 키워드.

scores.Sum()이 **마치 List의 메서드처럼** 호출됨.

List<int> 자체는 **건드리지 않았다**.

using으로 “보이게”만 한 **확장 메서드(extension method)**.

실험해 보기 (Lab 5)

- `scores.OrderByDescending(x => x).Take(3)`
→ 상위 3개만 출력
- `scores.Select(x => x * 2)`는 무엇을 만들까?
`.ToList()`로 새 List로 변환
- `List<int>`을 `List<string>`으로 바꿔서
`.OrderBy(s => s.Length)`로 정렬
→ 같은 LINQ가 어떤 타입에든

문제: “매 N초마다”를 자연스럽게

Update에서 타이머를 직접 굴리면...

- `timer -= Time.deltaTime;`
- `if (timer <= 0) { ...; timer = N; }`

동작은 하지만 시간 흐름의 의도가 가려진다.

→ `IEnumerator + yield return`

→ “만든다 → 기다린다 → 다시”를 일직선으로

Lab6_Coroutine.cs

```
1 using System;
2 using System.Collections;
3 using UnityEngine;
4
5 public class Lab6_Coroutine : MonoBehaviour
6 {
7     [SerializeField] private float interval = 1f;
8     private Coroutine running;
9
10    void Update()
11    {
12        if (Input.GetKeyDown(KeyCode.C))
13        {
14            if (running == null)
15                running = StartCoroutine(Tick(interval, n => Debug.Log($"Tick #{n}")));
16            else
17                { StopCoroutine(running); running = null; }
18        }
19    }
20
21    IEnumerator Tick(float wait, Action<int> onTick)
22    {
23        int count = 0;
24        while (true)
25        {
26            yield return new WaitForSeconds(wait);
27            count++;
28            onTick(count); // lambda invoked
29        }
30    }
31 }
```

IEnumerator + yield return

yield return X → 중단, X 돌려줌
호출자가 “계속해” → 중단된 곳부터 재개
Unity는 X에 따라 언제 재개할지 결정

n => Debug.Log(\$"Tick #{n}"): 이름 없는 함수

- n => : “n을 받는다”
- 본문: “이걸 한다”
- 함수를 값처럼 인자로 넘기는 사고방식
→ Action<int> 자리에 들어감

Lab 6 Unity 구현 가이드

- 1 Hierarchy → Create Empty → 이름 Lab6
- 2 Lab6_Coroutine.cs 작성, Lab6에 부착
- 3 Inspector의 Interval = 1
→ 0.5나 2로 바꿔 보기 (Play 중에도 가능)
- 4 Play → C 키 (시작/정지 토글)

i 관찰 포인트

Tick이 도는 동안에도
Unity의 다른 Update는 정상 동작.
Coroutine은 양보(yield)와 재개의 반복,
멀티스레드가 아니다.

기초 문법: 콜백 패턴

```
1 IEnumerator Tick(float wait, Action<int> onTick)
2 {
3     ...
4     onTick(count); // call the function we were given
5 }
```

“나중에 호출해 줘”라는 함수 인자 = 콜백(callback)

콜백 분리의 장점: Tick 그대로, “무엇을 할지”만 호출자가 결정.

```
1 StartCoroutine(Tick(1f, n => Debug.Log($"Tick {n}"))); // console
2 StartCoroutine(Tick(1f, n => labelText.text = $" {n}")); // UI
3 StartCoroutine(Tick(1f, n => audio.Play())); // sound
```

Lab 4 event와 같은 책임 분리 사상.

기초 문법: yield return이 돌려주는 “객체” 들

Coroutine 재개 시점은 **돌려주는 객체**로 결정된다.

- `yield return null;`
다음 프레임에 재개. 가장 자주.
- `yield return new WaitForSeconds(t);`
t초 후 재개 (timeScale 영향 받음).
- `yield return new WaitForSecondsRealtime(t);`
실제 시간 t초 후 재개.
- `yield return new WaitUntil(() => 조건);`
조건이 true될 때까지 대기.
- `yield return new WaitForEndOfFrame();`
그 프레임 렌더링 끝난 후.

“편지 내용”에 따라 Unity가 **언제 답장 줄지**를 결정.

실험해 보기 (Lab 6)

- `interval = 0.5f` 또는 `Time.timeScale = 0.1f`
- 콜백을 람다 안에서 분기:
 짝수 → `LogWarning`, 홀수 → `Log`
- `yield return new WaitForSeconds(wait)`을
 `yield return null`로 바꾸면?
 → 매 프레임마다 콜백.
 `yield return`이 무엇을 돌려주느냐가 핵심.

Lab	C# 기능
1	enum + switch 식
2	프로퍼티, =>
3	interface
4	event / Action<T>
5	제네릭 + LINQ + 람다
6	Coroutine + 람다

Unity는 우리에게 무엇을 해 주었나?

6개의 Lab은 모두 콘솔 C#으로도 만들 수 있었다.

하지만 Unity 위에서:

- enum이 큐브 색으로 바뀌어 보인다
- event가 색/위치/소리로 동시 반응
- Coroutine이 시간을 두고 펼쳐지는 것을 본다
- 키보드 입력으로 지금 당장 실험

★ 핵심 메시지

C#은 본질, Unity는 그것을 잘 보여 주는 무대

Phase 1 & 2

Phase 1 □ 같은 Lab에 한 줄 더

- Lab 1: `Mood.Excited` → 무지개색 큐브
- Lab 2: HP 슬라이더 (Lab 4의 이벤트 패턴 결합)

Phase 2 □ 두 Lab을 합치기

- “작은 인벤토리”: `IItem` + LINQ
(Lab 3 + Lab 5)
- “상태 기계”: `enum` + `event`
(Lab 1 + Lab 4)

Phase 3: 자유 실험

내가 가장 헛갈리는 C# 기능 한 가지를
스스로 Lab으로 만들어 본다.

추천 주제:

- struct vs class: 값 vs 참조
- out/ref 파라미터
- nullable int?와 ??
- abstract class vs interface
- static 멤버
- try/catch/finally

문제 → 콘솔 → Unity 데모 → 실험 과제
본인이 그 흐름을 작성해 본다.

이번 강의의 메시지

- C# 강의의 주인공은 C#
- Unity는 `Console.WriteLine`보다 친절한 무대
- 모든 C# 기능은 불편을 해결하는 도구
외워서 끼우는 것이 아니라 꺼내 쓰는 것

→ 다음 시간 예고

두세 가지 기능을 합쳐
조금 더 큰 Lab으로 발전시키기

질문은
언제든 환영합니다